StockSense AI — 1-Page Master Cheat Sheet

Stock Market Intelligence, Econometric ML Forecasting & Portfolio Analytics Platform

30-Second Elevator Pitch (Memorize This!):
"StockSense AI is a full-stack financial intelligence platform built with Python, Django, and Django REST Framework. It solves the problem of unreliable, fragmented trading dashboards by combining real-time NSE/US market ingestion, FIFO portfolio accounting, and honest econometric machine learning. Unlike black-box trading apps that invent fake 95% claims, my platform trains 5 distinct models—Linear Regression, Random Forest, ARIMA, Stacked LSTM, and a Hybrid Ensemble—on strictly chronological partitions with zero data leakage, validating performance transparently using MAE, RMSE, and Directional Accuracy."

Category	Verified Production Implementation in My Project
Core Stack	Python 3.13, Django 5.0.2, Django REST Framework 3.14.0, SQLite3 / PostgreSQL.
Modular Apps (8)	accounts (auth), market (prices), portfolio (FIFO ledger), watchlists, analytics (screener), predictions (ML lab), news (VADER), assistant (RAG copilot).
Database (12 Models)	UserProfile, Stock, HistoricalPrice, Portfolio, Transaction, Watchlist, WatchlistItem, PriceAlert, PredictionRun, NewsArticle, ChatConversation, ChatMessage.
5 ML Models	1) Linear Regression (AR lookback), 2) Random Forest (50 trees, max_depth=6), 3) ARIMA(1,1,0) on log returns, 4) Stacked LSTM (32-32-16-1, Dropout=0.15), 5) MSE-Optimized Hybrid Ensemble.
Zero Data Leakage	80% train / 20% test strictly chronological split (no shuffle). MinMaxScaler fit ONLY on training set and applied to test. Sliding window = 60 trading days.
Technical Indicators	Wilder's RSI(14), MACD(12,26,9), Bollinger Bands(20,2), SMA(20,50,200), EMA(12,26,50), ATR(14) computed vectorized via Pandas.
Performance & Cache	LocMemCache: 60s quote cache, 300s historical data cache. Vectorized batch test predictions in Keras.
Security & Auth	EmailOrUsernameModelBackend, PBKDF2-SHA256 (720k iterations), CSRF token verification, persistent database sessions (14 days).
Key URLs & Tests	71 registered URL routes across views and DRF endpoints. 19 automated tests in tests/test_platform.py (100% pass).

Top 3 Architectural Highlights for Interviewers:

- **Service Layer Pattern:** Views are lightweight controllers; all core logic resides in domain services (e.g., PortfolioAnalyticsService, ForecastingService) allowing decoupled unit testing.
- **Zero-N+1 Query Architecture:** All relational ORM lookups utilize select_related on ForeignKeys and prefetch_related on reverse sets, reducing database hits from O(N) to O(1).
- **Defensive Asynchronous Frontend:** Fetch calls inspect response.ok and Content-Type before parsing JSON, preventing SyntaxError: Unexpected token '<' on server errors or redirects.

Quantitative Finance & Machine Learning Formula Reference

Mathematical definitions, code implementations, and plain-English interview explanations

Relative Strength Index (RSI 14)	RSI = 100 - [100 / (1 + RS)], where RS = Average Gain / Average Loss over 14 periods via Wilder's smoothing.
Code:	`df['gain'] = delta.clip(lower=0); df['loss'] = -delta.clip(upper=0); rsi = 100 - (100 / (1 + roll_gain/roll_loss))`
Interview Takeaway:	Measures price momentum on a 0-100 scale. Readings > 70 suggest overbought conditions (potential pullback); readings < 30 suggest oversold conditions (potential bounce).
MACD (Moving Average Convergence Divergence)	MACD Line = EMA(12) - EMA(26); Signal Line = EMA(9) of MACD Line; Histogram = MACD Line - Signal Line.
Code:	`macd_line = close.ewm(span=12).mean() - close.ewm(span=26).mean(); signal = macd_line.ewm(span=9).mean()`
Interview Takeaway:	Trend-following momentum indicator. Bullish crossover occurs when MACD crosses above Signal; Bearish when it crosses below.
Bollinger Bands (20, 2)	Middle Band = 20 SMA; Upper Band = Middle + 2*sigma; Lower Band = Middle - 2*sigma (sigma = 20-period standard deviation).
Code:	`mid = close.rolling(20).mean(); std = close.rolling(20).std(); upper = mid + 2*std; lower = mid - 2*std`
Interview Takeaway:	Measures market volatility. Bands expand during high volatility and contract during consolidation ('the squeeze').
Mean Absolute Error (MAE)	MAE = (1/n) * sum(|y_true - y_pred|)
Code:	`mae = float(np.mean(np.abs(y_true - y_pred)))`
Interview Takeaway:	Measures the average magnitude of errors in raw rupee/dollar terms. Highly intuitive: if MAE is 25.40 on TCS, predictions deviate by Rs 25.40 on average.
Root Mean Squared Error (RMSE)	RMSE = sqrt[(1/n) * sum((y_true - y_pred)^2)]
Code:	`rmse = float(np.sqrt(np.mean((y_true - y_pred)**2)))`
Interview Takeaway:	Penalizes large outlier forecast errors more heavily than small ones because differences are squared before taking the root.
Directional Accuracy (%)	DA = [sum(sign(y_t - y_t-1) == sign(y_pred_t - y_t-1)) / n] * 100
Code:	`da = float(np.mean(np.sign(y_t[1:] - y_t[:-1]) == np.sign(y_pred[1:] - y_t[:-1])) * 100)`
Interview Takeaway:	Crucial financial metric! Answers: Did the model predict the correct direction (UP or DOWN)? A 55-60% directional accuracy in liquid markets is statistically significant.
Hybrid Ensemble Weight Optimization	y_pred = alpha * LSTM + (1 - alpha) * ARIMA; solve for argmin_alpha [MSE(alpha)] over alpha in [0.2, 0.8].
Code:	`best_alpha = min([0.2, 0.35, 0.5, 0.65, 0.8], key=lambda a: mean_squared_error(y_test, a*lstm + (1-a)*arima))`
Interview Takeaway:	Blends non-linear sequence memory (LSTM) with mean-reverting autoregression (ARIMA). Optimal alpha is found by grid search minimizing validation MSE.
FIFO Portfolio Cost Basis & Realized P&L	Position Cost = sum(Remaining Buy Lots * Buy Price); Realized P&L = (Sell Price - Matched Buy Price) * Quantity.
Code:	`Matched using a deque/queue where earliest BUY shares are relieved first on SELL orders.`
Interview Takeaway:	Enforces realistic tax accounting without distorting average costs when selling partial positions. Strict validation blocks overselling.

50 Lightning Revision Q&As; Across 7 Core Domains

Direct, concise model answers for technical interviews and viva examinations

Q1: What is StockSense AI and what problem does it solve?

It is a full-stack financial intelligence and econometric forecasting platform built with Django. It unifies fragmented stock quotes, technical analysis, VADER sentiment, FIFO portfolio tracking, and honest multi-model ML forecasting into a single, high-density terminal interface.

Q2: Why did you choose Django over Flask or FastAPI for this project?

Django provides an integrated ecosystem: a robust ORM, built-in cryptographic authentication, session management, CSRF security, and migration tooling out of the box. While Flask is minimal, Django's modular app structure allowed clean domain separation for 8 distinct applications.

Q3: How are the 8 Django apps organized?

Each app handles a single business domain: 'accounts' handles auth; 'market' handles quotes; 'portfolio' manages transactions and FIFO positions; 'watchlists' tracks user favorites; 'analytics' powers technical indicators and screeners; 'predictions' trains and evaluates ML models; 'news' analyzes sentiment; and 'assistant' runs the RAG copilot.

Q4: What is the service layer pattern in your codebase?

Django views only handle HTTP request parsing, user validation, and JSON/template response formatting. Business logic (such as portfolio cost basis calculations or ARIMA model training) is encapsulated in standalone service classes like PortfolioAnalyticsService and ForecastingService, making code testable and reusable.

Q5: What exchanges and symbols does the platform support?

It supports Indian National Stock Exchange equities (TCS.NS, INFY.NS, RELIANCE.NS, HDFCBANK.NS, SBIN.NS), major US equities (AAPL, MSFT, NVDA, GOOGL, TSLA), and market benchmark ETFs (SPY, QQQ, NIFTYBEES.NS).

Q6: How do you handle multi-currency display (INR vs USD)?

The UserProfile model stores base_currency (INR or USD). All transactions store their execution currency. When rendering portfolio metrics, prices are formatted with appropriate currency symbols and localized numeral formatting.

Q7: What happens if an external API like Yahoo Finance fails?

MarketDataService uses defensive fallbacks: it first checks the in-memory cache, then queries the local HistoricalPrice database table for the latest snapshot, and logs a warning rather than crashing the view.

Q8: How many automated tests exist in your project?

There are 19 comprehensive unit and integration tests in tests/test_platform.py covering user registration, dual login, portfolio trades, overselling prevention, watchlist management, and ML prediction endpoints. All pass with 100% success.

Q9: What is the difference between select_related and prefetch_related?

select_related performs an SQL JOIN and is used for single-valued relationships (ForeignKey, OneToOneField). prefetch_related performs a separate query with an IN filter and merges the results in Python, used for multi-valued relationships (ManyToManyField, reverse ForeignKey).

Q10: Where did you use select_related in StockSense AI?

In Transaction.objects.filter(portfolio=portfolio).select_related('stock'). This joins the Stock table in a single query, avoiding an extra SQL query for every transaction row when rendering portfolio holdings.

Q11: How do you enforce database transaction atomicity during trades?

I use django.db.transaction.atomic. When recording a trade, updating cash, or deducting shares, the operations execute inside a transaction context block. If an error occurs, all changes roll back, preventing partial or corrupted states.

Q12: How did you implement dual-identifier login (Username or Email)?

I implemented a custom authentication backend, EmailOrUsernameModelBackend, which overrides authenticate(). It queries User.objects.filter(Q(username__iexact=cred) | Q(email__iexact=cred)).first() and verifies the password using check_password().

Q13: Why did you register your custom backend in AUTHENTICATION_BACKENDS?

Django's default ModelBackend only checks username. By listing 'apps.accounts.backends.EmailOrUsernameModelBackend' first in AUTHENTICATION_BACKENDS, Django delegates authentication attempts to my custom class before falling back.

Q14: How do user sessions persist across development server restarts?

Django is configured with SESSION_ENGINE = 'django.contrib.sessions.backends.db'. Session tokens are stored in the database table 'django_session', so restarting the Python process does not destroy user sessions.

Q15: How does your code prevent overselling of stock shares?

Before executing a SELL transaction, PortfolioAnalyticsService aggregates all previous BUY and SELL quantities for that stock. If the sell quantity exceeds the currently owned quantity, it raises a ValidationError('Insufficient shares to execute sell').

Q16: How do you prevent Django CSRF verification failures in JavaScript fetch requests?

All POST, PUT, and DELETE fetch requests extract the CSRF token from the 'csrftoken' browser cookie or the DOM hidden input and attach it to the request headers as 'X-CSRFToken'.

Q17: What is data leakage in time-series forecasting?

Data leakage occurs when information from the future (the test set) is inadvertently used to train the model. Examples include random train/test shuffling, fitting scalers on the entire dataset, or using forward-looking rolling features.

Q18: How does StockSense AI guarantee zero data leakage?

Data is partitioned chronologically: the first 80% of historical dates form the training set, and the final 20% form the held-out test set. The MinMaxScaler is fit exclusively on the training set and applied to the test set.

Q19: What is the lookback sliding window size used in your models?

Lookback = 60 trading days (approximately 3 calendar months). The feature vector X_t contains close prices from [t-59 to t], predicting target y_t at t+1.

Q20: Why did you implement an Autoregressive Linear Regression baseline?

A linear regression model provides an interpretable benchmark. In complex financial modeling, if an expensive deep neural network cannot outperform a simple linear autoregression, the deep model is not justified.

Q21: How is Random Forest configured in your project?

RandomForestRegressor(n_estimators=50, max_depth=6, random_state=42). Restricting trees to 50 and depth to 6 prevents severe overfitting on noisy stock price sequences.

Q22: Why did you apply ARIMA on log returns rather than raw stock prices?

Raw stock prices are non-stationary (they exhibit trends and varying variance). Log returns, r_t = ln(P_t / P_t-1), are stationary and stabilize variance, satisfying the mathematical stationarity assumptions of ARIMA.

Q23: What is the neural network architecture of your Stacked LSTM?

Layer 1: LSTM(32 units, return_sequences=True); Layer 2: Dropout(0.15); Layer 3: LSTM(32 units, return_sequences=False); Layer 4: Dropout(0.15); Layer 5: Dense(16 units, relu); Layer 6: Dense(1, linear). Trained with Adam optimizer and MSE loss.

Q24: What is the purpose of Dropout layers in the LSTM?

Dropout randomly deactivates 15% of neuron activations during training batches. This prevents neurons from co-adapting to spurious market noise, acting as a powerful regularizer against overfitting.

Q25: How does the Hybrid LSTM-ARIMA Ensemble determine its weights?

It generates predictions as y_pred = alpha*LSTM + (1-alpha)*ARIMA. The service evaluates a candidate grid of alphas: [0.20, 0.35, 0.50, 0.65, 0.80] on the held-out validation set and selects the alpha that minimizes test MSE.

Q26: Why is Directional Accuracy more important than RMSE for traders?

A trader can profit if they know whether the market will move UP or DOWN, even if the exact price target is slightly off. RMSE only measures magnitude error, but Directional Accuracy directly measures trading signal fidelity.

Q27: How does yfinance integrate into your platform?

MarketDataService calls yf.Ticker(symbol).history(period='2y', interval='1d') to download daily open, high, low, close, and volume data. Data is returned as a Pandas DataFrame.

Q28: Why do you compute indicators using Pandas instead of looping through prices?

Pandas operations run on compiled C/Cython vector primitives via NumPy. Calculating a 200-day rolling mean across 1,000 bars takes under 1 millisecond vectorized, compared to 50-100ms with Python for-loops.

Q29: How does Wilder's Exponential Smoothing differ from standard EMA in RSI?

Standard EMA uses alpha = 2 / (N + 1), which gives higher weight to recent bars (alpha = 0.133 for N=14). Wilder's smoothing uses alpha = 1 / N (alpha = 0.0714 for N=14), producing a smoother, less jittery indicator.

Q30: What are the three components of MACD?

1. MACD Line: 12-day EMA minus 26-day EMA. 2. Signal Line: 9-day EMA of the MACD Line. 3. MACD Histogram: MACD Line minus Signal Line.

Q31: How do Bollinger Bands identify market volatility?

The band width is 4 standard deviations (upper minus lower). When volatility drops, the bands contract into a 'squeeze'. A subsequent expansion typically signals the start of a strong directional trend.

Q32: How is Average True Range (ATR) calculated?

True Range is max(High - Low, |High - Previous Close|, |Low - Previous Close|). ATR is the 14-period Wilder moving average of True Range, measuring absolute volatility in price units.

Q33: How does in-memory caching work in your project?

Django's LocMemCache caches ticker quote snapshots with a 60-second Time-To-Live (TTL) and 2-year historical dataframes with a 300-second TTL. Keys follow the format 'stock_quote_{symbol}'. Repeated requests return in < 5ms.

Q34: How do you handle stock symbol differences between NSE and US markets?

Indian National Stock Exchange symbols require the '.NS' suffix (e.g., TCS.NS, RELIANCE.NS), whereas US stocks do not (e.g., AAPL, MSFT). The Stock model stores exchange code ('NSE' vs 'NASDAQ'/'NYSE') and currency ('INR' vs 'USD').

Q35: How does Django store and verify passwords?

Django uses PBKDF2 with SHA-256 and 720,000 iterations. It generates a cryptographically secure random salt per user. Passwords are never stored in plain text. Verification hashes the input with the stored salt and compares.

Q36: What is a REST API and how does Django REST Framework implement it?

A REST API exposes resources over standard HTTP methods (GET, POST, PUT, DELETE) returning JSON. DRF provides APIView, ModelViewSet, and Serializers to validate incoming payloads and serialize Django model querysets into JSON.

Q37: What was the root cause of the 'SyntaxError: Unexpected token <' bug you fixed?

When the frontend called an invalid or unauthorized endpoint, Django returned an HTML 500 error page or a 302 login redirect. The JavaScript code called response.json() unconditionally. Parsing HTML '<!DOCTYPE html>' as JSON threw the SyntaxError.

Q38: How did you fix that error permanently?

Two-step fix: 1. Ensured the backend view returns JsonResponse({'success': False, 'error': msg}, status=400/500). 2. Updated frontend fetch to verify response.ok and headers.get('content-type').includes('application/json') before parsing.

Q39: How does StockSense AI prevent SQL injection?

All database queries use Django's ORM, which parameterizes all query values. User inputs are never concatenated directly into raw SQL strings.

Q40: How does your frontend protect against Cross-Site Scripting (XSS)?

Django templates automatically escape all variable outputs (e.g., {{ stock.name }}). In JavaScript, user-supplied text is inserted using textContent or safe DOM nodes rather than dangerous innerHTML concatenation.

Q41: What HTTP status codes do your API endpoints return?

200 OK for successful reads/forecasts; 201 Created for new trades/alerts; 400 Bad Request for validation errors; 401 Unauthorized for unauthenticated requests; 404 Not Found for missing stocks; and 500 Internal Error for unexpected backend failures.

Q42: How does the Plotly financial chart render dynamically?

The Django view returns historical dates, open, high, low, close, and volume as JSON arrays. The frontend Plotly.js engine builds a Candlestick trace and Bar volume trace, rendering an interactive canvas with crosshair tooltips.

Q43: What is VADER and why is it suited for stock news?

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a rule-based sentiment model tailored for microtext and financial headlines. It recognizes capitalization, exclamation marks, and polarity shifts without requiring GPU infrastructure.

Q44: What are the thresholds for VADER sentiment classification?

Compound score >= 0.05 is POSITIVE (Bullish); score <= -0.05 is NEGATIVE (Bearish); scores between -0.05 and +0.05 are NEUTRAL.

Q45: What is Retrieval-Augmented Generation (RAG) in your AI Copilot?

Instead of generating generic chatbot responses, ResearchAssistantService retrieves the user's live portfolio holdings, current stock prices, and 14-day RSI/MACD signals from the database and injects them as factual context into the response generator.

Q46: How are chat conversations stored in the database?

The ChatConversation model tracks user sessions. Each interaction generates two ChatMessage records: role='user' with the inquiry, and role='assistant' with the response and timestamp, allowing multi-turn memory.

Q47: How is the application deployed to production?

Production deployment utilizes Gunicorn as the WSGI application server behind an Nginx reverse proxy. Static files are served via WhiteNoise or Nginx. The database transitions from SQLite to PostgreSQL via DATABASE_URL.

Q48: How do you run the automated test suite?

python manage.py test tests.test_platform. It executes 19 tests in an isolated in-memory test database, validating all models, views, and APIs in ~5 seconds.

Q49: What is the most technically challenging problem you solved in this project?

Implementing the zero-data-leakage LSTM-ARIMA hybrid pipeline. Synchronizing the differenced stationary ARIMA predictions with the scaled multi-step sequence outputs of the LSTM while preventing forward-looking scaler contamination required meticulous pipeline architecture.

Q50: What would you improve if given 3 more months?

1. Add Celery with Redis for asynchronous background model training and WebSocket price updates. 2. Implement Walk-Forward Hyperparameter Optimization with Bayesian search. 3. Integrate broker OAuth APIs (Zerodha Kite or Alpaca) for paper trading execution.

Emergency Interview Survival Guide: What to Do When Stuck

Battle-tested strategies for handling tough questions, edge cases, and architectural challenges

Scenario 1: The interviewer asks a deep question about an ML math proof you don't know.
Say this: 'While I don't have the exact closed-form derivation memorized offhand, I understand its operational mechanism and trade-offs. In my project, I implemented it using [statsmodels/Keras] because... In production, my focus was ensuring zero data leakage and walk-forward validation.'
Why it works: Redirects from academic trivia to production engineering and empirical validation.

Scenario 2: The interviewer asks: 'Why didn't you use Celery for async training?'
Say this: 'That is an excellent architectural point. For the current deployment scale, running inference synchronously within a 2-second budget using vectorized Keras batching kept operational complexity low. In a commercial scale-up with 10,000 concurrent users, I would offload model execution to Celery workers backed by a Redis message queue and stream updates via WebSockets.'
Why it works: Shows you know when to avoid premature optimization while demonstrating full knowledge of distributed systems.

Scenario 3: The interviewer asks: 'Can your ML model guarantee profitable trading?'
Say this: 'No, and I explicitly engineered platform disclaimers against that assumption. Financial markets exhibit non-stationarity, regime shifts, and low signal-to-noise ratios. My models capture short-term autoregressive tendencies and price autocorrelation, but they cannot anticipate unforeseen geopolitical shocks. In finance, ML is a probabilistic risk-management tool, not a crystal ball.'
Why it works: Instantly distinguishes you from amateur candidates who claim fake 99% accuracy.

Scenario 4: The interviewer asks: 'Why use SQLite in development instead of PostgreSQL?'
Say this: 'SQLite requires zero external server setup and runs in-process, making local onboarding and CI/CD test runs lightning-fast. However, Django's database abstraction layer makes switching to PostgreSQL seamless: we simply update the DATABASES setting in settings.py with psycopg2. SQLite is for development agility; PostgreSQL is for concurrent production workloads.'
Why it works: Demonstrates practical engineering pragmatism and mastery of Django's database abstraction.

Scenario 5: The interviewer spots a potential flaw or asks how you would handle 100,000 transactions.
Say this: 'That's a great scaling challenge! In my current implementation, FIFO calculation runs in Python memory across past lots. For 100,000 historical transactions, that would cause latency. I would solve this by snapshotting monthly closed positions into an AccountSummary table, so the FIFO engine only iterates over the current active tax year's open lots.'
Why it works: Demonstrates algorithmic maturity and system design optimization capabilities.